

Parallelized Dynamic Programming for Nonlinear Macroeconomic Models: A MATLAB MEX-CUDA Approach

Alessandro Di Nola*

University of Birmingham

Liang Shi†

University of Essex

July 2026

*Department of Economics, University of Birmingham, Edgbaston, Birmingham, B15 2TT, United Kingdom.
E-mail: a.dinola@bham.ac.uk

†Institute for Social and Economic Research (ISER), University of Essex, Wivenhoe Park, CO4 3SQ, Colchester, United Kingdom. E-mail: shiliang369econ@gmail.com

Abstract

We present a MATLAB-centered approach that compiles nonlinear macroeconomic model code into C and CUDA MEX executables. The method reduces runtime and the coding burden of moving from MATLAB prototypes to compiled execution. Using a canonical one-period sovereign default model as the main benchmark, we show that MEX-CUDA delivers roughly an order-of-magnitude speedup relative to MATLAB-parfor while preserving the same MATLAB-centered workflow. The method remains effective in richer sovereign default and heterogeneous-agent applications, delivering full-solution speedups ranging from 10 to 30 relative to MATLAB-serial. The incremental gains relative to MEX-parfor are smaller because serial equilibrium components remain outside the compiled kernel. MEX-CUDA also remains competitive with CUDA Fortran and PyTorch-CUDA. The results show that MEX methods can relax computational constraints in applied macroeconomic research, while performance gains depend on the parallelizable structure of the underlying algorithm.

Keywords: Dynamic programming, GPU computing, MATLAB MEX, Nonlinear macroeconomics

JEL Codes: C63, C88, F41

1 Introduction

Quantitative research using nonlinear dynamic macroeconomic models often entails solving high-dimensional dynamic programming problems. As models incorporate richer state spaces, discrete choices, occasionally binding constraints, default risk, and heterogeneous agents, computational cost can constrain model design, calibration, and robustness analysis. The burden is particularly severe for iterative methods that update value, policy, pricing, or distributional objects over finely discretized domains. Grid-based value function iteration (VFI) is a leading example and remains useful when non-concavities, discontinuities, and discrete choices limit the applicability of methods relying on smooth first-order conditions or local approximations.

This paper studies a practical approach for relaxing this constraint. We show how computational kernels written in MATLAB can be compiled into C and CUDA executables through MATLAB Executables (MEX files), allowing researchers to retain a transparent MATLAB-centered workflow while moving the most time-consuming parts of iterative dynamic model solution to compiled CPU or GPU execution. We use a canonical one-period sovereign default model as the main benchmark because it is a familiar nonlinear macroeconomic environment in which grid-based VFI is widely used.¹ We then examine richer applications, including a long-maturity sovereign default model and a heterogeneous-agent model, to evaluate whether the same approach remains effective as model structure and computational intensity increase.

The paper makes two contributions. First, we provide a practical MATLAB-centered workflow for compiling performance-critical dynamic-programming routines into CPU and GPU executables. The approach allows researchers to retain MATLAB for model construction, calibration, and analysis while compiling loop-based computational kernels for execution through C or CUDA. It therefore differs from approaches that require rewriting the model in a lower-level language or moving the research workflow to a different programming environment. Previous work has used MEX primarily as an interface for calling externally written compiled code (Richter et al., 2014), while Kirkby (2017) develops the VFI Toolkit, which uses vectorized MATLAB commands for GPU parallelization. In contrast, we focus on compiling MATLAB functions containing the model’s loop-based core computations directly into MEX executables.

¹Replication code is available in the authors’ GitHub repositories.

Second, we evaluate this workflow across economically meaningful nonlinear macroeconomic models and alternative solution algorithms. In the canonical sovereign default model of Arelano (2008), we compare loop-based VFI implementations using brute-force search, binary-monotonicity search (Gordon and Qiu, 2018), and interpolation methods used in sovereign default applications (Hatchondo et al., 2010), with a fully vectorized implementation. We then consider a long-maturity sovereign default model with debt renegotiation and multiple exogenous shocks (Chatterjee and Eyigungor, 2012; Yue, 2010; Mihalache, 2020), and the heterogeneous-agent framework of Buera and Shin (2013). These exercises show how algorithmic and model structure shape both the scope and limits of parallelization. They also position the MATLAB-to-MEX-CUDA workflow relative to recent GPU-based work in PyTorch (Duarte et al., 2020), Julia (Deng et al., 2022), CUDA Fortran, and specialized-hardware approaches (Peri, 2020).

These comparisons connect the paper to the computational economics literature on parallelization. Early contributions include Aldrich et al. (2011) and Aruoba and Fernández-Villaverde (2015), while Fernández-Villaverde and Valencia (2018) anticipates the growing use of hybrid computational methods. Recent work studies GPU-based implementations in PyTorch (Duarte et al., 2020) and Julia (Deng et al., 2022), as well as specialized hardware such as field-programmable gate arrays (Peri, 2020). We complement this literature by identifying where a MATLAB-to-MEX-CUDA workflow is effective, where it is limited, and whether it remains competitive with CUDA Fortran and PyTorch-CUDA in comparable GPU benchmarks.

2 A MEX method for parallelization

MEX enables MATLAB to execute compiled code (e.g., C/CUDA) directly from the MATLAB environment. Using the `codegen` command, MATLAB Coder translates MATLAB functions into C or CUDA and compiles them into MEX executables. Online Appendix B illustrates the MEX setup on both local computers and a representative cloud service, Amazon Web Services (AWS). As shown in Listing 1, users can isolate the most computationally intensive component, such as the VFI routine, into a standalone MATLAB function and generate a corresponding MEX executable.

MEX accelerates execution relative to scripted MATLAB code by optimizing low-level oper-

ations and eliminating interpretation overhead. Vectorization, e.g. using `gpuArray`, can sometimes deliver similar gains, but often entails higher memory usage and reduced flexibility relative to loop-based implementations.

It is important to note that GPUs are not drop-in replacements for CPUs; fully exploiting their computational capacity typically requires code adjustments and compilation. MEX-CUDA can nevertheless deliver substantial performance gains with limited modifications to standard MATLAB code. The central question is therefore not whether GPU hardware is fast in general, but which parts of an iterative macroeconomic algorithm can be exposed as parallel computational kernels and what code restructuring is required to do so.

2.1 When does the method work?

The usefulness of MEX-CUDA depends on the computational structure of the model solution algorithm. The method is most effective when the costly part of the computation consists of many independent updates over state-grid points, candidate choices, or simulated agents. This structure is common in iterative macroeconomic algorithms that repeatedly update value, policy, pricing, or distributional objects over large state spaces. In these cases, a MEX-CUDA implementation can assign independent Bellman-update calculations, indexed by states and endogenous grid points, to GPU threads while retaining MATLAB for model setup, calibration, and post-solution analysis.

An important practical implication is that the performance-critical part of a dynamic programming algorithm need not be fully vectorized. In many problems, a loop-based implementation evaluates one state-choice calculation at a time and avoids constructing temporary payoff arrays for all states and choices simultaneously. Once compiled, these loops no longer incur MATLAB interpretation overhead and their independent iterations can be executed in parallel on the GPU. MEX-CUDA is therefore particularly useful when full vectorization would require large temporary arrays, impose substantial memory demands, or make model-specific logic difficult to express.²

The gains are more limited when the main bottleneck is inherently serial, when each parallel

²This tradeoff is familiar in high-performance computing: the attainable performance of a computational kernel can be bounded by memory bandwidth and data movement, not only by peak arithmetic throughput (Wulf and McKee, 1995; Williams et al., 2009; Jones, 2021).

task is too small to offset kernel-launch and data-transfer costs, or when the relevant operation is already handled by optimized linear-algebra libraries. Examples include low-dimensional problems with too few grid points to saturate the GPU, outer equilibrium price-search routines, convergence checks, and the relatively serial binary-monotonicity method (Gordon and Qiu, 2018). By contrast, root-finding and interpolation can benefit from MEX-CUDA when many independent instances are performed across the state grid, as in the interpolation experiment below.

Dense matrix operations are highly parallelizable and can often be dispatched to optimized BLAS or CUDA BLAS routines that are already highly parallelized. Overall, the main target for custom MEX-CUDA kernels is therefore model-specific operations such as Bellman maximization, feasibility checks, branching, and policy-index updates.

MEX-CUDA also does not remove the curse of dimensionality: it reduces the runtime of parallelizable operations, but the number of states, choices, and equilibrium objects still determines the size of the problem. The benchmarks below are therefore designed not only to report speedups, but also to show how algorithmic structure shapes the scope and limits of parallelization.

2.2 Generating MEX executables from MATLAB code

```

1 %% Generate MEX C brute-force VFI solver
2 cfg = coder.config('mex'); % configuration
3 cfg.GenerateReport = true; % code generation report
4 codegen -config cfg solver_bruteforce -args {b,z,m,pdf,para} ...
5     -o solver_bruteforce_mex % generate mex C function
6 % Call MEX C function
7 [q,bp,vp,def,totaltime,avgtime] = solver_bruteforce_mex(b,z,m,pdf,para);
8
9 %% Generate MEX CUDA brute-force VFI solver
10 cfg = coder.gpuConfig('mex'); % configuration
11 cfg.GenerateReport = true; % code generation report
12 codegen -config cfg solver_bruteforce_cuda -args {b,z,m,pdf,para} ...
13     -o solver_bf_cuda_mex % generate mex CUDA function
14 % Call MEX CUDA function
15 [q,bp,vp,def,totaltime,avgtime] = solver_bf_cuda_mex(b,z,m,pdf,para);

```

Listing 1: MATLAB code generating MEX functions for the brute-force VFI solver in the one-period sovereign default model. z and m are vectors for transitory and trend shocks, b is the vector of debt grid points, pdf is the joint transition matrix, and $para$ is a structure array for parameters. `cfg.GenerateReport = true` delivers a code-generation report for diagnostics purposes.

Listing 1 illustrates the basic code-generation workflow. The researcher first isolates the performance-critical routine, such as VFI, as a standalone MATLAB function and specifies its inputs for code generation. Input dimensions may be fixed or declared variable-size, depending on whether grid sizes and shock discretizations will vary across exercises.³ For GPU execution, the routine should avoid unnecessary temporary arrays and use compiler directives to identify the loops and memory operations that can be placed in GPU kernels. These adjustments are substantially lighter than writing explicit kernels (as in CUDA and Julia), while still allowing MATLAB functions to compile into GPU-executable MEX files.⁴

2.3 A benchmark Bellman kernel

Listing 2 illustrates the main MEX-CUDA adjustment in the benchmark one-period model. The MATLAB code keeps the brute-force search structure, but uses GPU Coder directives to expose nested loops to GPU parallelization. The directive `coder.gpu.kernelfun` marks the solver for GPU code generation, and `coder.gpu.kernel()` asks GPU Coder to parallelize the following nested loops. In the code, `ns` and `nb` denote the numbers of exogenous states and debt grid points, respectively.

The code deliberately combines matrix operations and nested loops. Matrix multiplications are used to form conditional expectations that are reused within the Bellman update and, in the CUDA-targeted MEX implementation, can be dispatched to optimized CUDA BLAS routines. For example, `pdf*vp` and `pdf*(theta*vo + (1-theta)*vd)` compute conditional expectations over next-period exogenous states. The line `w = b' .* q + exp(z).*m` is instead an element-wise matrix operation: it uses MATLAB implicit expansion to compute resources before subtracting current debt.

The reason for using nested loops in the maximization step is different. The expectation and resource calculations create only $N_s \times N_b$ intermediate arrays, whereas a fully vectorized maximization block would construct a payoff array with dimensions $N_s \times N_b \times N_b$, storing payoffs

³Variable-size inputs are useful because researchers often change grid sizes, shock discretizations, and calibration dimensions across robustness exercises; without this flexibility, a MEX file generated for one set of input dimensions must be regenerated when those dimensions change. The tradeoff is that flexible dimensions may reduce compile-time specialization. Online Appendix C reports a robustness check showing that variable-size inputs preserve most of the MEX-CUDA speed gains.

⁴Similarly, CUDA Fortran allows users to add annotations to standard Fortran code to generate efficient CUDA kernels.

```

1 function [q,bp,vp,def,totaltime,avgtime] = ...
2     solver_bruteforce_cuda(b,z,m,pdf,para)
3 coder.gpu.kernelfun; % mark function for GPU code generation
4 ...
5 evp = m.^(1-sigg).*beta.*pdf*vp; % expected repayment value
6 vd1 = ua + m.^(1-sigg).*beta.*pdf*(theta*vo + ...
7     (1-theta)*vd); % expected default value
8 w = b' .* q + exp(z).*m; % resources before current debt
9
10 coder.gpu.kernel() % parallelize following loops on the GPU
11 for is = 1:ns
12     for ib = 1:nb
13         tmpmax = -Inf; maxidx = 1;
14         for i = 1:nb
15             c = w(is,i) - b(ib);
16             if c > 0
17                 u = 1 - 1/c + evp(is,i);
18             else
19                 u = -Inf;
20             end
21             if u > tmpmax
22                 tmpmax = u; maxidx = i;
23             end
24         end
25         vp1(is,ib) = tmpmax; bp(is,ib) = maxidx;
26     end
27 end

```

Listing 2: Benchmark one-period brute-force VFI kernel used to generate MEX-CUDA code. The code computes the repayment value over future debt choices without forming large temporary arrays inside the GPU kernel.

for every state, current debt level, and future debt choice at once. The algorithm does not need to retain this full array: for each state and current debt level, it requires only the maximum payoff and the index of the maximizing future-debt choice. The innermost loop therefore evaluates one candidate choice at a time, computing a scalar consumption value and payoff and updating the scalar temporary variables `tmpmax` and `maxidx`. This avoids large temporary arrays, and exposes independent state-by-debt maximization problems to the GPU.

The benchmark kernel also clarifies the scale and limits of the acceleration. Parallelization does not change the asymptotic complexity of grid-based dynamic programming, but it can substantially reduce runtime when many Bellman calculations are independent. In the brute-force sovereign default benchmark, each state-by-current-debt pair evaluates candidate future debt choices independently, exposing roughly $N_s \times N_b$ independent maximization problems, each involving N_b candidate choices. CPU-based `parfor` parallelization distributes only the N_s outer-loop iterations across a limited number of CPU workers, whereas the CUDA kernel exposes the

larger set of $N_s \times N_b$ calculations, indexed by $(\mathbf{is}, \mathbf{ib})$ in Listing 2, to GPU threads.

The attainable speedup depends on the share of total runtime represented by this parallelizable block, as well as on operations that are not fully parallelized, including price updates, convergence checks, CPU–GPU data transfers, and branch handling inside the objective function. MEX-CUDA can therefore substantially reduce the runtime of the Bellman-update block, while the remaining components continue to shape total solution time.⁵

3 Experiments

This section evaluates the performance of MEX-based implementations in solving nonlinear macroeconomic models. We begin with a canonical one-period sovereign default model because it provides a transparent and conservative benchmark. We then study richer model structures: a long-maturity sovereign default model with debt renegotiation and a heterogeneous-agent model with asset and productivity heterogeneity. Additional model details for the richer applications are provided in Online Appendix D.

3.1 Canonical one-period sovereign default benchmark

3.1.1 Model and calibration

The benchmark model follows the one-period defaultable debt structure in Arellano (2008), augmented with cyclical and trend endowment shocks (Aguiar and Gopinath, 2006). Let $\mathbf{s} = (z, \gamma)$ denote the exogenous state vector and let b denote outstanding debt. The government chooses whether to repay or default:

$$V(b, \mathbf{s}) = \max \{ V^p(b, \mathbf{s}), V^d(\mathbf{s}) \}. \quad (1)$$

The value of repayment is

$$V^p(b, \mathbf{s}) = \max_{c \geq 0, b'} u(c) + \tilde{\beta} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} V(b', \mathbf{s}'), \quad (2)$$

⁵We discuss this point further in the experiment of the heterogeneous-agent model.

subject to

$$c = y + \gamma q(b', \mathbf{s})b' - b, \quad (3)$$

where $y = \exp(z)\gamma$ and $\tilde{\beta} \equiv \beta\gamma^{1-\sigma}$. The value of default is

$$V^d(\mathbf{s}) = u((1 - \psi)y) + \tilde{\beta}\mathbb{E}_{\mathbf{s}'|\mathbf{s}} [(1 - \mu)V^d(\mathbf{s}') + \mu V^p(0, \mathbf{s}')], \quad (4)$$

and competitive foreign lenders price debt according to

$$q(b', \mathbf{s}) = \frac{1}{1 + r^*} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} [1 - d(b', \mathbf{s}')]. \quad (5)$$

Utility is CRRA, $u(c) = (c^{1-\sigma} - 1)/(1 - \sigma)$, with $\sigma = 2$. The cyclical shock z and trend shock γ follow AR(1) processes:

$$z_t = \rho_z z_{t-1} + \varepsilon_t^z, \quad \varepsilon_t^z \sim \text{i.i.d. } \mathcal{N}(0, \sigma_z^2), \quad (6)$$

$$\gamma_t = (1 - \rho_\gamma)\bar{\gamma} + \rho_\gamma \gamma_{t-1} + \varepsilon_t^\gamma, \quad \varepsilon_t^\gamma \sim \text{i.i.d. } \mathcal{N}(0, \sigma_\gamma^2). \quad (7)$$

The benchmark calibration sets $\beta = 0.85$, $r^* = 0.01$, the default output loss to $\psi = 0.028$, the reentry probability to $\mu = 0.0385$, $\rho_z = \rho_\gamma = 0.90$, $\sigma_z = 0.030$, $\sigma_\gamma = 0.008$, and $\bar{\gamma} = 1.02$. We discretize each exogenous shock into 25 grid points using the Tauchen method with width 4.0, producing $N_s = 625$ exogenous states. The VFI loop terminates when the maximum absolute differences in V^p , V^d , and q fall below 10^{-7} .

3.1.2 Implementations and runtime results

The benchmark solves this model by brute-force VFI on a common discretized debt grid: both the current debt state b and the next-period debt choice b' take values in the same N_b -point grid. We also evaluate whether MEX gains survive under three common implementation changes: vectorized computation, binary monotonicity, and linear interpolation. In all cases, debt grid is evenly spaced on $b \in [0, 1.01]$, with debt-grid sizes reported in Table 1.

Binary monotonicity exploits the monotonicity of the optimal debt choice with respect to the debt state (Gordon and Qiu, 2018). With N_b debt states and N_b debt choices, it re-

duces the number of objective function evaluations from N_b^2 under brute-force search to at most $N_b \log_2(N_b) + 5N_b$. Hence, monotonicity reduces the number of debt-choice evaluations, whereas parallelization accelerates the remaining evaluations without eliminating the curse of dimensionality.

The linear interpolation method evaluates both $q(b', \mathbf{s})$ and $\mathbb{E}[V(b', \mathbf{s}')] off the debt grid. Hatchondo et al. (2010) show that interpolation methods can solve one-period sovereign default models accurately with substantially fewer debt grid points. In our implementation, interpolation and root-finding are loop-intensive and involve conditional branching, which makes this case useful for assessing whether MEX gains persist in branch-heavy code. The vectorized brute-force implementation instead forms larger temporary arrays, which reduces explicit loop overhead but increases memory pressure.$

Here, *MATLAB-serial* refers to loop-based VFI written in native MATLAB, while *MATLAB-parfor* differs only by the use of the `parfor` construct. *MEX-serial* and *MEX-parfor* compile the same routines using MEX for execution on a multi-core CPU, and *MEX-CUDA* offloads computation to the GPU. *MATLAB-gpuArray* transfers arrays to GPU memory using MATLAB’s `gpuArray` syntax and then applies vectorized GPU operations, in line with the approach used in the VFI Toolkit of Kirkby (2017). *MEX-CUDA-vec* applies MEX compilation to the same *MATLAB-gpuArray* implementation.

Table 1 reports results on two local computing environments. The laptop computer uses an Intel Core i7-11800H CPU and an NVIDIA RTX 3060 laptop GPU, while the desktop computer uses an Intel Core Ultra 265K CPU and an NVIDIA RTX 5070 GPU. The laptop provides a baseline for a relatively accessible research workstation, whereas the desktop represents newer local hardware. Full hardware and software details are provided in Online Appendix B.

All reported runtimes (in seconds) follow a common benchmarking protocol. One-time code-generation and computing-environment initialization costs are excluded. Each implementation is executed six times: the first execution serves as a warm-up and is discarded, and the tables report the arithmetic mean over the remaining five executions. Before each measured CUDA execution, `wait(gpuDevice)` is called to ensure that no previously queued GPU work remains, and elapsed time is recorded only after the generated VFI routine has completed and returned its outputs. The CUDA timing includes CPU–GPU data movement performed within the compiled

VFI function. Within each algorithmic comparison, all implementations use identical initial conditions and stopping rules and complete the same number of iterations.

Table 1: Runtime (seconds) for the canonical one-period sovereign default model

Panel A. Loop-based algorithms					
Methods	Brute-force		Binary monotonicity		Linear interpolation
	$N_b = 200$	$N_b = 400$	$N_b = 200$	$N_b = 400$	$N_b = 80$
1. Laptop computer					
MATLAB-serial	129.5	429.5	5.4	11.8	129.7
MATLAB-parfor	26.1	68.5	11.6	15.2	36.0
MEX-serial	61.6	180.3	26.0	46.9	41.5
MEX-parfor	21.6	57.4	24.9	44.0	313.6
MEX-CUDA	2.0	6.9	3.0	5.7	2.8
2. Desktop computer					
MATLAB-parfor	13.5	31.4	4.3	6.0	6.7
MEX-parfor	4.8	13.3	5.0	10.0	9.9
MEX-CUDA	1.1	3.3	1.2	2.3	0.81
Panel B. Fully vectorized brute-force VFI					
Methods	$N_b = 200$		$N_b = 400$		
1. Laptop computer					
MATLAB-vec	74.0		287.0		
MATLAB-gpuArray	12.3		427.9		
MEX-CUDA-vec	9.68		43.1		
2. Desktop computer					
MATLAB-gpuArray	6.6		27.6		
MEX-CUDA-vec	4.8		17.1		

Notes: Number of grids for exogenous shocks is $N_s = N_z \times N_\gamma = 625$ in all cases. N_b denotes the number of debt grid points. In Panel A, N_b also denotes the number of debt choices for brute-force and binary monotonicity. The desktop panels focus on parallel implementations; MATLAB-serial, MEX-serial, and MATLAB-vec are therefore not reported for that computer. All GPU computations use double precision.

Table 1 shows that the MEX-CUDA implementation is consistently faster than the corresponding MATLAB implementations in the canonical model. In the brute-force case with $N_b = 400$, MEX-CUDA reduces runtime from 68.5 seconds under MATLAB-parfor to 6.9 seconds on the laptop computer, a speedup of 9.9. The desktop computer further reduces runtimes because of newer CPU and GPU hardware, but the qualitative ranking of methods remains the same.

The alternative algorithms show that MEX-CUDA gains are not confined to the brute-force

benchmark. Binary monotonicity reduces the number of objective-function evaluations in the search problem, so its absolute runtimes are much lower than brute-force VFI. At the same time, it introduces a more serial control structure, which limits additional gains from CPU parallelization.⁶ These results also show that CPU-based MEX gains are not automatic: they depend on how well the algorithm maps into compiled loops and CPU parallel execution.

Linear interpolation provides a different performance test. On the laptop, MATLAB-`parfor` improves substantially on serial MATLAB but remains far slower than MEX-CUDA, while MEX-`parfor` performs poorly. The interpolation routine still contains many independent optimization problems across the state grid, so MEX-CUDA remains effective despite its conditional branching, interpolation, and root-finding operations.

Panel B provides a useful contrast between full vectorization and loop-based MEX-CUDA kernels. Although vectorized GPU implementations improve substantially on native MATLAB-vectorized code, they fall behind the loop-based MEX-CUDA implementation in Panel A. On the laptop computer with $N_b = 400$, for example, MEX-CUDA-vec takes 43.1 seconds, compared with 6.9 seconds for loop-based MEX-CUDA. This result echoes the implementation discussion above: forming large temporary payoff arrays can slow execution. The loop-based MEX-CUDA kernel instead evaluates candidate choices one at a time and keeps only the running maximum and policy index. As the debt grid grows, the memory cost of the vectorized method becomes more severe: it exhausts the 6 GB GPU memory on the laptop computer when $N_b > 500$, whereas the loop-based MEX-CUDA implementation can handle up to $N_b = 10,000$ in the same model.

These differences matter for economic research because calibration, sensitivity analysis, and model debugging require repeated solutions of the same model under alternative parameter values and grid choices. The laptop brute-force benchmark with $N_b = 400$, for example, takes 68.5 seconds under MATLAB-`parfor` and 6.9 seconds under MEX-CUDA, implying roughly an order-of-magnitude increase in the number of model solves that can be completed in a fixed research time budget. The memory comparison points in the same direction: avoiding large temporary arrays expands the feasible grid size and therefore the scope for robustness checks.

⁶Gordon and Qiu (2018) also discuss the difficulty of parallelizing binary monotonicity and related divide-and-conquer methods across CPU cores.

3.1.3 Comparison with other high-performance environments

Potential users may also ask whether MEX is merely faster than MATLAB or whether it is competitive with established high-performance alternatives. To address this question, Table 2 reports two comparison exercises on the laptop computer. Panel 1 compares MEX with Fortran-OpenMP and CUDA Fortran using the same one-period sovereign default model and brute-force implementation as Table 1. The CUDA Fortran implementation uses OpenACC directives with the NVIDIA HPC SDK to compile array operations and annotated Fortran loops into GPU-executable CUDA kernels. Like MEX-CUDA, this approach avoids the need to write explicit low-level GPU kernels, which can be technically demanding in CUDA (Aldrich et al., 2011) and Julia (Deng et al., 2022). Fortran-OpenMP provides the corresponding CPU-parallel comparison using the OpenMP API.

Panel 2 uses the vectorized sovereign default benchmark of Duarte et al. (2020), which is well suited for comparison with PyTorch. PyTorch is a Python-based framework that has become one of the most widely used GPU-computing libraries. Here, PyTorch-CPU denotes execution on the CPU, while PyTorch-CUDA uses JIT to compile the VFI routines and offloads the resulting operations to the GPU.

Table 2: Runtime (seconds) across selected high-performance environments

Methods	$N_b = 500$	$N_b = 1000$
<i>1. One-period debt model</i>		
MEX-parfor	78.2	205.8
Fortran-OpenMP	76.5	201.3
MEX-CUDA	9.1	26.0
CUDA-Fortran	9.4	26.8
<i>2. Duarte et al. (2020) model</i>		
MATLAB-vec	101.4	403.3
PyTorch-CPU	191.7	861.5
MEX-CUDA-vec	31.9	124.7
PyTorch-CUDA	30.6	119.4

Notes: Both panels use the same debt-grid sizes. Panel 1 uses the one-period sovereign default model and brute-force implementation from Table 1, with $N_s = 625$ exogenous shock states. CUDA Fortran is benchmarked on Ubuntu 24.04 LTS on the same laptop computer because the NVIDIA HPC SDK/CUDA Fortran compiler is not available on Windows; all other experiments use Windows 11. Panel 2 uses the vectorized sovereign default model from Duarte et al. (2020), which has 256 exogenous shock states. The panels therefore differ in model structure, shock-grid size, and implementation style. All implementations use double precision.

The results show that MEX-CUDA delivers runtime similar to CUDA Fortran in the one-period sovereign default model, while MEX-CUDA-vec remains close to PyTorch-CUDA in the vectorized benchmark.⁷ Because the two panels differ in model structure, shock-grid size, and implementation style, the cross-panel levels should not be interpreted as a direct ranking of loop-based and vectorized approaches. Instead, the table shows that the corresponding MEX-CUDA implementation is competitive with established compiled and GPU-based alternatives within each benchmark.

3.2 Richer model structures

We next ask whether the same approach continues to work when the model structure becomes more demanding. The long-maturity sovereign default model extends the canonical benchmark by incorporating long-duration debt, debt renegotiation, and multiple exogenous shocks (Chatterjee and Eyigungor, 2012; Mihalache, 2020; Kuang and Shi, 2026). The heterogeneous-agent model follows Buera and Shin (2013) and features asset and productivity heterogeneity with occupational choice, a framework widely used to study entrepreneurship and financial frictions (see, for example, Buera et al., 2015; Buera et al., 2021; Morazzoni and Sy, 2022).

The heterogeneous-agent application is a useful stress test for two reasons. First, the occupational choice margin creates a non-concavity in the value function, making straightforward VFI attractive relative to more delicate Euler-equation-based methods.⁸ Second, the reported runtime covers the full stationary-equilibrium solve, including price search, stationary distribution iteration, and aggregation. Since only the VFI block, including construction of the state- and choice-specific return matrix, is compiled or parallelized, this application shows how much acceleration remains when important equilibrium components are left outside the kernel.

Table 3 shows that MEX gains remain sizable in richer model structures, although their magnitude depends on the comparison baseline. At the larger grid sizes on the laptop computer,

⁷Unlike Duarte et al. (2020), who benchmark PyTorch and TensorFlow using single precision (FP32) but employ double precision (FP64) for other languages, we use FP64 consistently across all implementations. This choice makes the comparison conservative for GPU-oriented frameworks, since FP32 can be much faster than FP64 on consumer GPUs. We use FP64 because numerical accuracy is important in quantitative macroeconomic models. In our sovereign default applications, FP32 can lead to failed convergence or solutions that differ noticeably from the FP64 benchmark.

⁸Fella (2014) shows how to generalize the endogenous gridpoint method (EGM) to non-concave problems. As Gortz and Mirza (2018) point out, however, generalized EGM is more complex to implement than standard VFI.

Table 3: Runtime (seconds) for richer model structures

Methods	Long-maturity sovereign default		Heterogeneous-agent model	
	$N_b = 200$	$N_b = 400$	$N_a = 1001$	$N_a = 1501$
<i>1. Laptop computer</i>				
MATLAB-serial	101.6	372.5	190.7	265.2
MATLAB-parfor	3077	15988	1129	6556
MEX-serial	53.1	185.9	136.2	238.7
MEX-parfor	12.0	49.7	29.6	51.7
MEX-CUDA	4.7	13.6	15.2	22.9
<i>2. Desktop computer</i>				
MATLAB-parfor	33.1	57.9	907.8	1426
MEX-parfor	3.0	9.5	11.6	17.7
MEX-CUDA	1.7	4.5	4.1	5.5

Notes: N_b denotes the number of debt grid points in the sovereign default model; N_a denotes the number of asset grid points in the heterogeneous-agent model. The desktop panel focuses on parallel implementations; MATLAB-serial and MEX-serial are therefore not reported for that computer. GPU computations are conducted in double precision. For the heterogeneous-agent model, reported runtimes cover the full solve, including general-equilibrium price search and stationary distribution iteration. Across implementations, only the VFI block, including construction of the return matrix, is parallelized or compiled into MEX; the price search, distribution iteration, and aggregation steps remain in MATLAB.

MEX-CUDA delivers speedups of 27.4 relative to MATLAB-serial in the long-maturity sovereign default model and 11.6 in the heterogeneous-agent model. Relative to MEX-parfor, which isolates the incremental gain from moving an already compiled CPU-parallel routine to the GPU, the corresponding speedups are 3.7 and 2.3. The smaller full-model gain in the heterogeneous-agent application is informative because important equilibrium components remain outside the compiled VFI block.⁹ The smaller full-solve speedup is therefore consistent with these components limiting the attainable acceleration.

The poor laptop performance of MATLAB-parfor is also informative: at both grid sizes it is substantially slower than MATLAB-serial, requiring 15,988 seconds for the long-maturity model at $N_b = 400$. By contrast, MATLAB-parfor performs much better on the newer desktop computer. This comparison indicates that CPU-worker parallelization can be highly sensitive to processor architecture, worker count, memory performance, and scheduling overhead. Compiled MEX code provides more consistent acceleration by reducing interpretation and worker-

⁹In auxiliary experiments, compiling the price-search, stationary-distribution, and aggregation routines into MEX did not materially reduce total runtime.

management overhead, while MEX-CUDA additionally exploits finer-grained GPU parallelism. The results therefore show that adding MATLAB parallel syntax does not by itself guarantee faster execution. Compiled MEX code reduces interpretation overhead and gives the compiler greater scope to optimize the performance-critical VFI block, while MEX-CUDA additionally exploits finer-grained GPU parallelism.

3.3 Numerical agreement between CPU and GPU solutions

Because MEX-CUDA changes the execution environment rather than the economic algorithm, we also check whether the GPU and corresponding CPU implementations deliver the same numerical solutions. Table 4 reports sup-norm differences between CPU and MEX-CUDA solution objects across the main experiments.

Table 4: Numerical agreement between CPU and MEX-CUDA solutions

Model / algorithm	Grid	Value	Price	Policy	Other
<i>1. Canonical one-period sovereign default model</i>					
Brute force	$N_b = 200$	$< 10^{-14}$	0	0	0
Brute force	$N_b = 400$	$< 10^{-14}$	0	0	0
Binary monotonicity	$N_b = 200$	$< 10^{-14}$	0	0	0
Binary monotonicity	$N_b = 400$	$< 10^{-14}$	0	0	0
Interpolation	$N_b = 80$	1.46×10^{-8}	0	1.38×10^{-7}	0
<i>2. Richer model structures</i>					
Long-maturity sovereign default	$N_b = 200$	$< 10^{-14}$	$< 10^{-13}$	0	0
Long-maturity sovereign default	$N_b = 400$	$< 10^{-14}$	$< 10^{-13}$	0	0
Heterogeneous-agent model	$N_a = 1001$	$< 10^{-13}$	0	0	0
Heterogeneous-agent model	$N_a = 1501$	$< 10^{-12}$	0	0	0

Notes: Entries report maximum absolute differences between corresponding CPU and MEX-CUDA solution objects; entries reported as $< 10^{-k}$ are smaller than the displayed order of magnitude. CPU refers to the non-CUDA implementation of the same algorithm. For the sovereign default models, “Value,” “Price,” “Policy,” and “Other” correspond to the value function, bond price, debt policy, and default decision, respectively. For the heterogeneous-agent model, they correspond to the value function, equilibrium price vector, policy rule, and stationary distribution, respectively. All CPU and GPU computations use double precision.

In the grid-search cases, the CPU and MEX-CUDA implementations agree up to differences consistent with accumulated double-precision floating-point rounding. The interpolation case produces somewhat larger but still small discrepancies: the maximum policy difference is approximately 1.38×10^{-7} , while the bond prices and default decisions are identical across

implementations. Thus, each MEX-CUDA implementation reproduces the solution of its corresponding CPU algorithm to the reported numerical tolerances. These diagnostics establish agreement across execution environments.

4 Conclusion

This paper presents a hybrid approach for solving nonlinear quantitative economic models in which MATLAB code is compiled into C/CUDA MEX executables and executed within MATLAB. Using a canonical one-period sovereign default model as a benchmark, we show that MEX-CUDA delivers robust speedups across brute-force, vectorized, binary monotonicity, and interpolation-based implementations. The long-maturity sovereign default and heterogeneous-agent applications show that substantial gains remain in richer full-model solves, but also that the incremental GPU advantage becomes smaller when serial equilibrium components account for a larger share of runtime. By reducing both runtime and the coding burden of moving from MATLAB prototypes to compiled CPU/GPU execution, MEX helps researchers relax computational constraints and focus on richer model design. The results therefore clarify both the scope and the limits of the approach: MEX-CUDA reduces the runtime cost of parallelizable computations, but algorithmic structure, memory constraints, and serial steps continue to shape total runtime and feasible model size.

Acknowledgements

We thank Christoph Görtz, Juan Francisco Rubio-Ramirez, Oliver de Groot, Robert Kirkby and anonymous referees for their comments. Competing interests: The authors declare none. All errors are our own.

References

Aguiar, M. and Gopinath, G. (2006). Defaultable debt, interest rates and the current account. *Journal of International Economics*, 69(1):64–83.

- Aldrich, E. M., Fernández-Villaverde, J., Gallant, A. R., and Rubio-Ramírez, J. F. (2011). Tapping the supercomputer under your desk: Solving dynamic equilibrium models with graphics processors. *Journal of Economic Dynamics and Control*, 35(3):386–393.
- Arellano, C. (2008). Default risk and income fluctuations in emerging economies. *American Economic Review*, 98(3):690–712.
- Aruoba, S. B. and Fernández-Villaverde, J. (2015). A comparison of programming languages in macroeconomics. *Journal of Economic Dynamics and Control*, 58:265–273.
- Buera, F. J., Fattal-Jaef, R. N., Hopenhayn, H., Neumeyer, P. A., and Shin, Y. (2021). The economic ripple effects of covid-19. NBER Working Papers 28704, National Bureau of Economic Research, Inc.
- Buera, F. J., Kaboski, J. P., and Shin, Y. (2015). Entrepreneurship and financial frictions: A macrodevelopment perspective. *Annual Review of Economics*, 7(1):409–436.
- Buera, F. J. and Shin, Y. (2013). Financial frictions and the persistence of history: A quantitative exploration. *Journal of Political Economy*, 121(2):221–272.
- Chatterjee, S. and Eyigungor, B. (2012). Maturity, indebtedness, and default risk. *American Economic Review*, 102(6):2674–99.
- Deng, M., Guerron-Quintana, P. A., and Tseng, L. (2022). Parallel computation of sovereign default models. *Computational Economics*, pages 1–39.
- Duarte, V., Duarte, D., Fonseca, J., and Montecinos, A. (2020). Benchmarking machine-learning software and hardware for quantitative economics. *Journal of Economic Dynamics and Control*, 111:103796.
- Dvorkin, M., Sánchez, J. M., Sapriza, H., and Yurdagul, E. (2021). Sovereign debt restructurings. *American Economic Journal: Macroeconomics*, 13(2):26–77.
- Fella, G. (2014). A generalized endogenous grid method for non-smooth and non-concave problems. *Review of Economic Dynamics*, 17(2):329–344.
- Fernández-Villaverde, J. and Valencia, D. Z. (2018). A practical guide to parallelization in economics. Technical report, National Bureau of Economic Research.
- Gordon, G. and Qiu, S. (2018). A divide and conquer algorithm for exploiting policy function monotonicity. *Quantitative Economics*, 9(2):521–540.
- Gortz, C. and Mirza, A. (2018). Solving models with jump discontinuities in policy functions. *Oxford Bulletin of Economics and Statistics*, 80(2):434–456.
- Hatchondo, J. C., Martinez, L., and Sapriza, H. (2010). Quantitative properties of sovereign default models: solution methods matter. *Review of Economic Dynamics*, 13(4):919–933.
- Jones, S. (2021). How gpu computing works. In *GPU Technology Conference Digital April*. NVIDIA.
- Kirkby, R. (2017). A toolkit for value function iteration. *Computational Economics*, 49(1):1–15.
- Kuang, P. and Shi, L. (2026). Sovereign debt pricing with shifting long-run growth expectations. *European Economic Review*, 186(105310).

- Mihalache, G. (2020). Sovereign default resolution through maturity extension. *Journal of International Economics*, 125:103326.
- Mihalache, G. (2025). Solving default models. *Oxford Research Encyclopedia of Economics and Finance*, 0.
- Morazzoni, M. and Sy, A. (2022). Female entrepreneurship, financial frictions and capital misallocation in the us. *Journal of Monetary Economics*, 129:93–118.
- Peri, A. (2020). A hardware approach to value function iteration. *Journal of Economic Dynamics and Control*, 114:103894.
- Richter, A. W., Throckmorton, N. A., and Walker, T. B. (2014). Accuracy, speed and robustness of policy function iteration. *Computational Economics*, 44:445–476.
- Tauchen, G. (1986). Finite state markov-chain approximations to univariate and vector autoregressions. *Economics Letters*, 20(2):177–181.
- Williams, S., Waterman, A., and Patterson, D. (2009). Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76.
- Wulf, W. A. and McKee, S. A. (1995). Hitting the memory wall: Implications of the obvious. *ACM SIGARCH Computer Architecture News*, 23(1):20–24.
- Yue, V. Z. (2010). Sovereign default and debt renegotiation. *Journal of International Economics*, 80(2):176–187.

Online Appendix

Parallelized Dynamic Programming for Nonlinear Macroeconomic Models: A MATLAB MEX-CUDA Approach

by Alessandro Di Nola, Liang Shi

July 2026

A Supplementary data

Supplementary MATLAB code can be found in authors' GitHub repositories.

B Computation platform

B.1 Local computing environments

Our benchmark results are obtained on two local computing environments: a laptop computer and a desktop computer. The software requirements differ between generating MEX files and executing already generated MEX files. To generate MEX files, the user needs a MATLAB licence with the relevant code-generation toolboxes and a compatible C/C++ compiler; if MEX-CUDA is generated, an NVIDIA GPU and compatible GPU driver are also required. To execute an already generated MEX-CUDA file inside MATLAB, the target machine does not need MATLAB Coder, GPU Coder, a host compiler, or a CUDA Toolkit installation, but it must use a compatible operating system, MATLAB release, NVIDIA GPU, and GPU driver/runtime environment. The laptop computer used in the benchmark experiments has the following hardware and software:

1. Intel Core i7-11800H CPU with 8 physical cores;
2. Dynamic Random Access Memory (DRAM) of 16 GB operating at 3200 MHz;

3. NVIDIA GeForce RTX 3060 GPU (Laptop version) with 6 GB GPU memory (GRAM) and 30 streaming multiprocessor (SM) counts;
4. MATLAB 2023b with Parallel Computing Toolbox, MATLAB Coder, and GPU Coder;
5. Microsoft Windows 11 Pro operating system;
6. GPU driver: NVIDIA GeForce Game Ready Driver, Release 580 Driver for Windows, Version 581.29;
7. Code-generation setup: Microsoft Visual Studio 2019 for MEX compilation. CUDA Toolkit 12.0 was also installed for standalone CUDA experiments and cross-platform rebuilds, but it is not required merely to execute generated MEX-CUDA files inside MATLAB. See *Supported and Compatible Compilers for GPU Coder* website maintained by the MathWorks.

The desktop computer used in the benchmark experiments has the following hardware:

1. Intel Core Ultra 265K CPU with 20 physical cores;
2. NVIDIA GeForce RTX 5070 GPU with 12 GB GRAM and 48 SM counts;

The desktop uses MATLAB 2026a and Microsoft Visual Studio 2022 for code generation targeting the RTX 5070 GPU. All other software prerequisites are the same as on the laptop computer unless otherwise noted.

For the CPU-parallel experiments, MATLAB-parfor uses a local pool of eight workers on the laptop computer and 20 workers on the desktop computer, corresponding to the number of physical CPU cores. Each MATLAB worker uses one computational thread. MEX-parfor uses eight CPU threads on the laptop and 20 CPU threads on the desktop, while the Fortran-OpenMP comparison uses eight threads on the laptop. MATLAB-serial and MEX-serial use no explicit outer-loop parallelization, although MATLAB’s default implicit multithreading remains enabled for optimized linear-algebra operations. Thus, “serial” refers to the implementation structure rather than strict single-core execution.

B.2 Applying MEX on clouds

Beyond single-device parallelization on a CPU or GPU, MEX also supports distributed execution across clusters and cloud platforms such as Amazon Web Services (AWS), Microsoft Azure, or institutional high-performance computing (HPC) systems. This capability makes MEX methods practical for economists tackling large-scale nonlinear models. Although the benchmark results reported in the main text use local hardware, the same workflow can be deployed on AWS for larger-scale experiments.

To build an environment that can generate MEX files directly on AWS, we launch an EC2 instance and install MATLAB with the required toolboxes, GPU drivers, and third-party compilers. Users can create a custom Amazon Machine Image (AMI) based on the environment of this instance, which can be reused to deploy additional instances.¹⁰ While users can generate MEX files directly on AWS, they may also compile them locally and upload them to an AWS instance for execution. This approach eliminates the need to install MATLAB Coder, GPU Coder, third-party compilers, or CUDA Toolkit on AWS, provided that the target machine has a compatible MATLAB installation, operating system, NVIDIA GPU, and GPU driver/runtime environment. It can, however, forgo device-specific optimizations that code generation on the target machine can provide.

C Robustness to variable-size inputs

Variable-size inputs are of practical interest because applied researchers rarely solve a model only once at a single grid size. During calibration, sensitivity analysis, and debugging, users may repeatedly change the number of debt grid points, asset grid points, or shock states. Fixed-size MEX inputs allow the compiler to specialize code for a particular problem dimension, but they require code regeneration when those dimensions change. Variable-size inputs, enabled through the `coder.varsizes` pragma, allow one compiled MEX function to accept a range of input dimensions up to declared upper bounds.

Table C.1 panel 1 reports the corresponding fixed-size-input results: the sovereign default

¹⁰MathWorks' official GitHub page (MathWorks Reference Architectures) also offers pre-built AMIs ready for deployment on major commercial clouds.

results are copied from Table 1, while the heterogeneous-agent results are copied from Table 3. On our laptop computer, code generation takes approximately three seconds for MEX-serial and MEX-parfor, and eight seconds for MEX-CUDA. Panel 2 shows that allowing variable-size inputs results in negligible performance loss for MEX-CUDA, but more noticeable slowdowns for MEX-serial and MEX-parfor.

Table C.1: Runtime (seconds) for benchmark models, variable-size inputs

Methods	One-period sovereign default		Heterogeneous-agent model	
	$N_b = 200$	$N_b = 400$	$N_a = 1001$	$N_a = 1501$
<i>1. Fixed-size inputs</i>				
MEX-parfor	21.6	57.4	29.6	51.7
MEX-CUDA	2.0	6.9	15.2	22.9
<i>2. Variable-size inputs</i>				
MEX-parfor	34.1	76.5	42.7	102.8
MEX-CUDA	2.3	7.3	16.0	23.4

Notes: N_b denotes the number of debt grid points in the sovereign default model; N_a denotes the number of asset grid points in the heterogeneous-agent model. For the sovereign default model, “variable size” allows $N_b \in [1, 1000]$ and $N_s \in [1, 1000]$. For the heterogeneous-agent model, the asset grid is declared variable size with $N_a \in [1, 3001]$, while the productivity grid is fixed at $N_z = 40$. Experiments are conducted on the laptop computer.

D Details of benchmark models

D.1 Long-maturity sovereign default model

The sovereign default model builds on the long-maturity debt structure in (Chatterjee and Eyigungor, 2012), combined with debt renegotiation (Mihalache, 2020), and both transitory and trend endowment shocks as in Aguiar and Gopinath (2006). The model uses taste shocks (Mihalache, 2020; Dvorkin et al., 2021; Mihalache, 2025) to introduce convergence in the long-term debt framework with discrete choices. Such a model exemplifies the complexity in modern quantitative modeling, but still in small-to-moderate scale.

We denote $\mathbf{s} = (z, \gamma)$ as a vector of cyclical and trend endowment shocks, and $\tilde{\beta} \equiv \beta\gamma^{1-\sigma}$ as the trend-adjusted discount factor. In this economy, a benevolent government decides whether

to repay or default:

$$V(b, \mathbf{s}) = \max_{d \in \{0,1\}} (1-d)V^p(b, \mathbf{s}) + dV^d(\mathbf{s}), \quad (\text{D.1})$$

where V^p is the value of repaying debts:

$$V^p(b, \mathbf{s}) = \max_{c \geq 0, b'} u(c) + \tilde{\beta} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} V(b', \mathbf{s}'), \quad (\text{D.2})$$

subject to the household's budget constraint:

$$c = y + q[\gamma b' - (1-\lambda)b] - [\lambda + (1-\lambda)\eta]b, \quad (\text{D.3})$$

where $y = \exp(z)\gamma$ is the detrended exogenous endowment. V^d is the value of default:

$$V^d(\mathbf{s}) = u((1-\psi)y) + \tilde{\beta} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} \left[(1-\mu)V^d(\mathbf{s}') + \mu V^p(\Phi(\mathbf{s}'), \mathbf{s}') \right], \quad (\text{D.4})$$

where $1-\mu$ is the probability that the sovereign carries bad credit record and cannot issue new debts, while with μ probability it can renegotiate a new debt owed to creditors $\Phi(\mathbf{s}')$. ψ is the fraction of lost endowment due to default.

Regarding debt renegotiation, the sovereign's threat point is the value of autarky:

$$V^{aut}(\mathbf{s}) = u((1-\psi)y) + \tilde{\beta} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} V^{aut}(\mathbf{s}'), \quad (\text{D.5})$$

while creditors' highest loss is zero recovery ($\phi = 0$). Hence, the sovereign's surplus is

$$\Delta^s(\phi, \mathbf{s}) = V^p(\phi, \mathbf{s}) - V^{aut}(\mathbf{s}), \quad (\text{D.6})$$

and the creditors' surplus as

$$\Delta^c(\phi, \mathbf{s}) = \left[\lambda + (1-\lambda)(\eta + q(\phi', \mathbf{s})) \right] \phi, \quad (\text{D.7})$$

where $\phi' = \mathcal{B}(\phi, \mathbf{s})$ and $\mathcal{B}(\phi, \mathbf{s})$ is the borrowing policy from (D.2). The restructured debt (ϕ) is

determined by a Nash Bargaining game:

$$\Phi(\mathbf{s}) = \arg \max_{\phi} \left[\Delta^{\mathbf{s}}(\phi, \mathbf{s}) \right]^{\alpha} \left[\Delta^c(\phi, \mathbf{s}) \right]^{1-\alpha}, \quad \Delta^{\mathbf{s}} \geq 0 \text{ and } \Delta^c \geq 0. \quad (\text{D.8})$$

Foreign creditors take the expected default loss and recovery value into debt price:

$$q(b', \mathbf{s}) = \frac{1}{1+r^*} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} \left\{ (1-d(b', \mathbf{s}')) \left[\lambda + (1-\lambda)(\eta + q(b'', \mathbf{s}')) \right] + d(b', \mathbf{s}') q^d(b', \mathbf{s}') \right\}. \quad (\text{D.9})$$

where r^* is the risk-free rate, q^d is the price (recovery rate) for restructured bonds:

$$q^d(b', \mathbf{s}) = \frac{1}{1+r^*} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} \left\{ (1-\mu) q^d(b', \mathbf{s}') + \mu \frac{\Delta^c(\phi, \mathbf{s}')}{b} \right\}, \quad \phi = \Phi(\mathbf{s}') \quad (\text{D.10})$$

The one-period debt model discussed in Section 3.1 is a simplified version of this long-maturity model. Specifically, the repayment budget constraint becomes:

$$c = y + \gamma q b' - b, \quad (\text{D.11})$$

which is a special case of Equation (D.3) for $\eta = 0$ and $\lambda = 1$. Because the one-period model abstracts from debt restructuring ($\phi = 0$), the value of default becomes:

$$V^{d,one}(\mathbf{s}) = u((1-\psi)y) + \tilde{\beta} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} \left[(1-\mu) V^{d,one}(\mathbf{s}') + \mu V^p(0, \mathbf{s}') \right]. \quad (\text{D.12})$$

Without the value of restructured debt, the pricing rule becomes:

$$q^{one}(b', \mathbf{s}) = \frac{1}{1+r^*} \mathbb{E}_{\mathbf{s}'|\mathbf{s}} [1 - d(b', \mathbf{s}')], \quad (\text{D.13})$$

which is a special case of Equation (D.9) when $\lambda = 1$ and the price of restructured debt q^d does not exist.

Calibration of the long-maturity model is reported in Table D.1. To implement the VFI, we discretize the space of γ and z each into 25 equally spaced grid points, hence $N_s \equiv N_{\gamma} \times N_z = 625$. The probability transition matrix $\Pi(\mathbf{s}'|\mathbf{s})$ approximating $\mathbb{E}_{\mathbf{s}'|\mathbf{s}}$ is generated with the Tauchen (1986) method, as is standard in this literature. Outstanding debts b , debt policy b'

Table D.1: Calibration of the long-maturity sovereign default model

Description	Parameters	Value
Risk aversion	σ	2
Subjective discount	β	0.83
Probability of bargaining	μ	0.154
Endowment loss	ψ	10%
Coupon rate	η	0.06
Reciprocal of maturity	λ	0.15
Quarterly risk-free rate	r^*	4%
z shock AR(1)	ρ_z	0.90
SD of z shock	σ_z	3%
γ shock AR(1)	ρ_γ	0.90
SD of γ shock	σ_γ	0.75%
Average trend	$\bar{\gamma}$	1.01
Bargaining power	α	0.6
Debt issuance taste	σ^b	0.001
Default taste	σ^d	0.001
Renegotiation taste	σ^ϕ	0.001

Notes: Parameters target annual frequency.

and restructured debts ϕ share an equally spaced discretization within $[0.1, 1.4]$. The number of debt grids (N_b) takes 200 and 400, commonly used in this strand of literature.

D.2 Heterogeneous-agent model

The heterogeneous-agent application follows Buera and Shin (2013). Individuals differ in wealth a and entrepreneurial productivity z , and choose whether to work for a wage or operate a business. Productivity follows a finite-state Markov chain with transition matrix $Q(z, z')$.

For the runtime experiments, the compiled component is the value-function update for a given price vector (r, w) . The outer equilibrium routine, including price search with `fminsearch`, stationary distribution iteration, and aggregation of factor demands and asset supply, is kept unchanged across implementations. This design isolates the effect of compiling the individual dynamic-programming problem while reporting runtimes for the full stationary-equilibrium solve.

D.2.1 Model and computation

For given prices (r, w) , the individual problem is

$$V(a, z) = \max_{c, a'} \left\{ \frac{c^{1-\gamma}}{1-\gamma} + \beta \mathbb{E} \left[V(a', z') \mid z \right] \right\}, \quad (\text{D.14})$$

subject to

$$c + a' \leq \max \left\{ w, \pi(a, z \mid r, w) \right\} + (1+r)a, \quad a' \geq 0, \quad (\text{D.15})$$

where $\pi(a, z \mid r, w)$ denotes entrepreneurial profits:

$$\pi(a, z \mid r, w) = \max_{l, k} \{ f(z, k, l) - wl - (r + \delta)k \}, \quad \text{s.t.} \quad k \leq \lambda a, \quad (\text{D.16})$$

with production function

$$f(z, k, l) = z(k^\alpha l^{1-\alpha})^{1-\nu}, \quad (\text{D.17})$$

where $\nu \in (0, 1)$ is the span-of-control parameter. The occupational choice is

$$e(a, z) = \begin{cases} 1, & \text{if } \pi(a, z \mid r, w) > w, \quad (\text{entrepreneur}) \\ 0, & \text{otherwise.} \quad (\text{worker}) \end{cases} \quad (\text{D.18})$$

The entrepreneur's static problem has a closed-form solution:

$$k^{unc}(z) = \left[\left(\frac{\alpha(1-\nu)z}{r+\delta} \right)^{1-(1-\alpha)(1-\nu)} \left(\frac{(1-\alpha)(1-\nu)z}{w} \right)^{(1-\alpha)(1-\nu)} \right]^{1/\nu}, \quad (\text{D.19})$$

$$k(a, z) = \min \{ k^{unc}(z), \lambda a \}, \quad (\text{D.20})$$

$$l(a, z) = \left[\frac{(1-\alpha)(1-\nu)z}{w} (k(a, z))^{\alpha(1-\nu)} \right]^{\frac{1}{1-(1-\alpha)(1-\nu)}}. \quad (\text{D.21})$$

$$y(a, z) = z(k(a, z)^\alpha l(a, z)^{1-\alpha})^{1-\nu}, \quad (\text{D.22})$$

$$\pi(a, z \mid r, w) = y(a, z) - wl(a, z) - (r + \delta)k(a, z). \quad (\text{D.23})$$

We solve the stationary equilibrium by nesting VFI inside an outer price search. For a candidate (r, w) , the static factor demands are precomputed, the Bellman problem is solved

with Howard improvement, and the stationary distribution $\mu(a, z)$ is obtained from the savings policy and the exogenous productivity transition. Prices are updated by minimizing squared excess demands,

$$ED^K(r, w) = \int k(a, z) \mathbb{I}_{\{e(a, z)=1\}} d\mu(a, z) - \int a d\mu(a, z), \quad (\text{D.24})$$

$$ED^L(r, w) = \int l(a, z) \mathbb{I}_{\{e(a, z)=1\}} d\mu(a, z) - \int \mathbb{I}_{\{e(a, z)=0\}} d\mu(a, z), \quad (\text{D.25})$$

using MATLAB's `fminsearch` routine.

D.2.2 Calibration and discretization

We follow the calibration strategy in Buera and Shin (2013). The externally set parameters are the coefficient of relative risk aversion $\gamma = 1.5$, depreciation $\delta = 0.06$, and capital share $\alpha = 0.33$. The remaining parameters are set to $\beta = 0.904$, $\nu = 0.21$, $\eta = 4.15$, $\psi = 0.894$, and $\lambda = \infty$, corresponding to the U.S. benchmark in Buera and Shin (2013). The general-equilibrium price search starts from $r = 0.0476$ and $w = 0.172$.

The asset grid covers $a \in [10^{-6}, 4000]$ with a quadratic spacing that places more points near zero. The runtime experiments in Table 3 use $N_a = 1001$ and $N_a = 1501$ asset grid points. The variable-size experiments in Online Appendix C compile the asset grid as a variable-size input with an upper bound of $N_a = 3001$.

Entrepreneurial ability z follows a persistent stochastic process with a truncated Pareto innovation distribution,

$$p(z) = \begin{cases} \eta z^{-(\eta+1)}, & z \geq 1, \\ 0, & \text{otherwise.} \end{cases} \quad (\text{D.26})$$

Each period, an individual keeps z with probability ψ and draws a new $z' \sim p(\cdot)$ with probability $1 - \psi$. Let the discretized support be $z \in \{z_1, z_2, \dots, z_{N_z}\}$, and let $p = [p_1, \dots, p_{N_z}]'$ denote the associated probability vector. In the computations, the productivity support has $N_z = 40$ grid points. The finite-state transition matrix used in the VFI and distribution iteration is

$$\Pi_{N_z \times N_z} = \psi \mathbf{I}_{N_z} + (1 - \psi) \mathbf{1}_{N_z} p^T, \quad (\text{D.27})$$

where $\mathbf{I}_{N_z}$ is the $N_z \times N_z$ identity matrix and $\mathbf{1}_{N_z}$ is an $N_z \times 1$ column vector of ones. For replication, we use the files `support.dat` and `dist.dat` kindly provided by Buera and Shin (2013).